

Case Study - Portfolio And Dividend Tracker

Martin Joo

Introduction

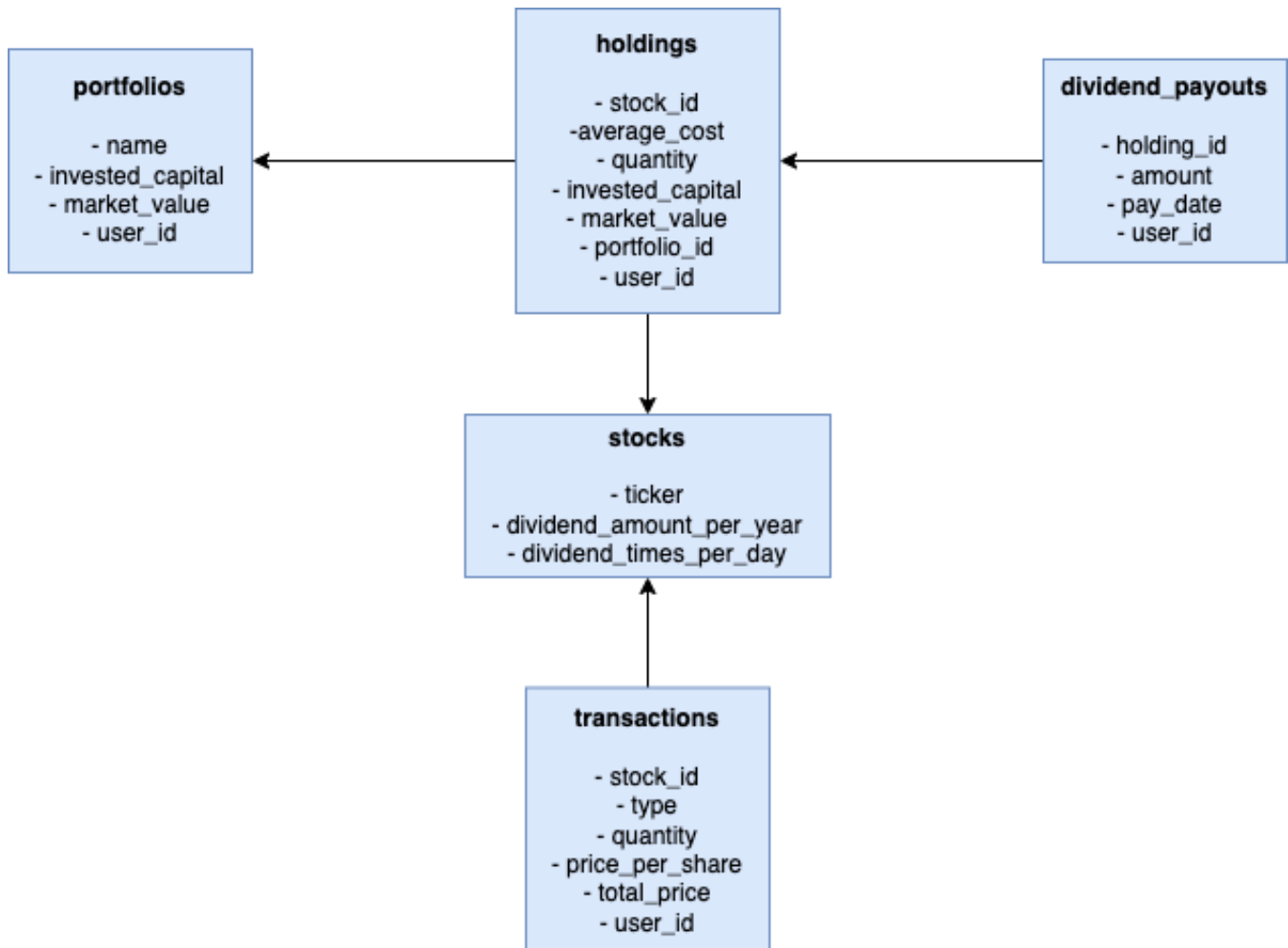
In this short case study, we will design a portfolio and dividend tracker application. First of all, let's define what the application does. It has four main features:

- An overview of your stock portfolios.
 - You can see the invested capital in each stock and portfolio.
 - The current market value of them.
 - And also yields such as dividend yield, or the overall yield (based on invested capital and current market value).
- A summary of received dividends.
 - How much money did you receive this week, this month, or all time?
- A monthly view of received dividends.
 - A quick summary of every month.
- A monthly statement of invested capital.
 - It shows you how much money you invested each month.

There are no CRUD actions in this system, so you cannot create a new holding or anything like that. All of the data is constructed by CSV imports. The market values are updated using a 3rd party API called Market Stack.

Data Modeling

It's a small application, but let's go through the database and models because it's important.



Let's go through these models.

Stock

It represents a publicly-traded company such as Apple. Each of these companies has a ticker symbol, such as AAPL. We often use the ticker symbol instead of the company name when talking about investments and portfolios.

In the context of a portfolio tracker, a stock is a lightweight model. As you can see, it only has three essential columns:

- `ticker`
- `dividend_amount_per_year` is the total \$ amount of the dividend paid by this company each year.
- `dividend_times_per_year` is a number that represents how many times this company distributes its dividend to the shareholders.

I will show you an example:

id	ticker	dividend_amount_per_year	dividend_times_per_year
1	AAPL	0.88	4
2	MSFT	2.48	4

The first row means Apple pays a \$0.88 dividend a year and it sends you a whopping \$0.22 each quarter so 4 times a year. In the case of Microsoft, they will reward you with \$2.48 a year or \$0.62 every quarter. These numbers apply to one share.

Now let's talk about the names.

- Why not call this table `companies`? At the end of the day, it contains information about publicly-traded companies. That's correct, however, I chose the name `stock` because it's very lightweight. I've been working on other financial applications and usually, a company means much more than this. It doesn't even have a name.
- As you can see, I'm very specific with the column names. It's not just `dividend_amount`, it's `dividend_amount_per_year`. It's because dividends are measured per year but are usually distributed quarterly. So it would be very ambiguous if it was called `dividend_amount`.

Holding

When an investor purchases a stock, it becomes their holding. You can think about it this way: there's only one Apple, but millions of Apple stocks are owned by individual investors. Each person invests a different amount of money into AAPL stock. Let me show you an example:

- You bought 1 AAPL share for \$100 one month ago.
- You purchased another AAPL share two weeks ago. But now the price was \$110.
- As of today, AAPL is trading at \$120.

After these actions, the holdings table would look like this:

id	stock_id	average_cost	quantity	invested_capital	market_value	user_id
1	1	105.00	2	210	120	1

You invested \$210 into Apple and bought two shares, so your average cost is \$105. The average cost is very important because this is how we can calculate the gains. In this example, you gained 14.28%.

- Your average cost is \$105
- One Apple share is currently worth \$120
- Your gain is \$15
- You gained \$15 on every share of yours, so your overall gain is
- $15/105 = 14.28\%$

As you can see, this table is the basis of every critical calculation. How does this table gets populated? By importing the transactions.

Transaction

Whenever you buy or sell a share, it's a transaction. And, of course, there's always a stock involved. There are several important attributes:

id	stock_id	type	quantity	price_per_share	total_price	user_id
1	1	buy	1	100	100	1
2	1	buy	4	110	440	1
3	1	sell	-1	120	-120	1

In this application, I decided to store the `total_price` and the `quantity` as negative numbers if the transaction is a sell. This is beneficial because it's extremely easy to sum them up:

```
$holding = new Holding();

// 1 + 4 + (-1) = 4
$holding->quantity = $transactions->sum('quantity');
// 100 + 440 + (-120) = 420
$holding->invested_capital = $transaction->sum('total_price');
```

As I said earlier, transactions become holdings when importing a CSV. So the three rows in the example will become one holding of AAPL. And with negative numbers, it's straightforward to calculate the properties of the holding. **This choice heavily depends on the concrete use-cases.** In this instance, it's a good idea because it makes things easier.

Dividend Payout

This table contains the individual dividend payments from your holdings. Here are some examples:

id	holding_id	amount	paid_at	user_id
1	1	0.2050	2021-02-11	1
2	1	0.2200	2021-05-10	1
3	1	0.2200	2021-08-12	1
4	1	0.2200	2021-11-11	1

These four rows contain the quarterly paid dividends of Apple in 2021. Each row belongs to a holding. It's important to note that it belongs to a holding, not a stock. So this example assumes that the holding's quantity is precisely 1. If you have 2 Apple shares instead, the numbers will double up. So instead of 0.2050, it would be 0.4100, and instead of 0.2200, it'd be 0.4400.

We can calculate the sum of dividends paid by a holding of yours from this table. So if your `average_cost` for Apple is \$100 and you got these dividends:

- Overall dividend paid: \$0.865
- Your yield on cost: $0.865/100 = 0.865\%$

Portfolio

A portfolio is just a container for holdings. It's not an essential model from a technical point of view, but it's important for users. A portfolio has two aggregate numbers:

- Invested capital
- Market value

Each of them equals the sum of the individual holdings' invested capital or market value.

Custom Collections

Weighted Average Price

Another remarkable aspect of Laravel I haven't talked about in the book is custom collections. In this application, I have transactions. At some point, transactions become holdings. Because of this, I need to do three things frequently when I work with a collection of transactions:

- Get the sum of `quantity`
- Get the sum of `total_price`
- Get the weighted average of `price_per_share`

Let's take the third as an example:

```
$transactions = Transaction::all();

$sumOfProducts = $transactions
    ->sum(fn (Transaction $transaction) =>
        $transaction->quantity * $transaction->price_per_share
    );

if ($transactions->sum('quantity') === 0.00) {
    return 0;
}

$weightedPricePerShare = $sumOfProducts / $transactions->sum('quantity');
```

Now, where do you put this code? Let's go through some options:

- Action. It can work if you only need it in one place.
- View model. The same applies here.
- Model. It solves the problem of reusability, so let's see how it looks like

```

class Transaction extends Model
{
    public static function weightedPricePerShare(
        Collection $transactions
    ): float {
        $sumOfProducts = $transactions
            ->sum(fn (Transaction $transaction) =>
                $transaction->quantity * $transaction->price_per_share
            );

        if ($transactions->sum('quantity') === 0.00) {
            return 0;
        }

        $weightedPricePerShare = $sumOfProducts / $transactions->sum('quantity');
    }
}

```

Now, my main problem is this: it has nothing to do with the transaction model. It's a static method; it does not use the `$this`, so, in my opinion, it doesn't belong to the `Transaction` model. My general rule: **if you have a static method in a model, you probably can find a better place for it.**

If you think about this snippet, it only exists in the context of a collection, right? Fortunately, in Laravel, we can write a custom collection for a model:

```
namespace App\Collections\Transaction;

use App\Models\Transaction;
use Illuminate\Database\Eloquent\Collection;

class TransactionCollection extends Collection
{
    public function weightedPricePerShare(): float
    {
        $sumOfProducts = $transactions
            →sum(fn (Transaction $transaction) ⇒
                $transaction→quantity * $transaction→price_per_share
            );

        if ($transactions→sum('quantity') ≡ 0.00) {
            return 0;
        }

        $weightedPricePerShare = $sumOfProducts / $transactions→sum('quantity');
    }
}
```

So you can define a custom `TransactionCollection` that extends Laravel's Collection class. The last step is to instruct Laravel to actually use this class whenever we're creating a new collection from `Transaction` models:

```
namespace App\Models;

use App\Collections\Transaction\TransactionCollection;
use Illuminate\Database\Eloquent\Factories\HasFactory;

class Transaction extends Model
{
    use HasFactory;

    public function newCollection(array $models = []): TransactionCollection
    {
        return new TransactionCollection($models);
    }
}
```

Now, here comes the real magic:

```
$holding = new Holding();

$transactions = Transaction::all();
$holding->price_per_share = $transactions->weightedPricePerShare();
```

And here's the important thing. A transaction doesn't have a weighted price per share. But a collection of transactions do have. **The example above expresses this perfectly.** So by using custom collections, we can write more expressive, domain-oriented code.

Since we want to sum everything when it comes to transactions, let's add some more methods to the collection class:

```

namespace App\Collections\Transaction;

use App\Models\Transaction;
use Illuminate\Database\Eloquent\Collection;

class TransactionCollection extends Collection
{
    public function sumQuantity(): float
    {
        return $this->sum('quantity');
    }

    public function sumTotalPrice(): float
    {
        return $this->sum('total_price');
    }

    public function weightedPricePerShare(): float
    {
        $sumOfProducts = $this
            ->sum(fn (Transaction $transaction) =>
                $transaction->quantity * $transaction->price_per_share
            );

        if ($this->sumQuantity() === 0.00) {
            return 0;
        }

        return $sumOfProducts / $this->sumQuantity();
    }
}

```

And finally, this is one part of the app that uses this collection:

```

class CreateHoldingsFromTransactions
{
  public function execute(
    TransactionCollection $transactions,
    User $user
  ): Collection {
    return $transactions
      →groupBy('stock_id')
      →map(fn (TransactionCollection $transactions, int $stockId) ⇒
        Holding::updateOrCreate(
          [
            'stock_id' ⇒ $stockId,
            'user_id' ⇒ $user→id,
          ],
          [
            'average_cost' ⇒ $transactions→weightedPricePerShare(),
            'quantity' ⇒ $transactions→sumQuantity(),
            'invested_capital' ⇒ $transactions→sumTotalPrice(),
            'ticker' ⇒ $transactions→first()→stock→ticker,
          ]
        )
      );
  }
}

```

It's very high-level, in my opinion. And it helps you to bring the code closer to the domain language, which is very important for me. You can apply this technique to a significant number of things.

Yield On Cost

The yield on cost (YoC) is the dividend payout divided by the average cost. Now, we can calculate YoC on a great number of things:

- An individual holding.
- A number of holdings.
- A whole portfolio.
- A number of portfolios together.

Let's see some examples. This is how the YoC is calculated on a holding:

```
class Holding extends Model
{
  public function getYieldOnCostAttribute(): float
  {
    if (!$this->stock->dividend_amount_per_year) {
      return 0;
    }

    return $this->stock->dividend_amount_per_year / $this->average_cost;
  }
}
```

For example, if Apple pays \$0.865 every year and my average cost is \$100, this will return 0.0865 or 0.865%. With a custom collection, we can also calculate the YoC on a number of holdings:

```

namespace App\Collections\Holding;

use App\Models\Holding;
use Illuminate\Database\Eloquent\Collection;

class HoldingCollection extends Collection
{
    public function yieldOnCost(): ?float
    {
        $sumOfProducts = $this
            ->sum(fn (Holding $holding) =>
                $holding->yield_on_cost * $holding->invested_capital
            );

        if ($sumOfProducts === 0.0) {
            return null;
        }

        return $sumOfProducts / $this->sum('invested_capital');
    }
}

```

This is the weighted average of YoCs. However, in real life, there's no such thing as "weighted average of yield on costs"; it's just yield on cost. This is why I'm using the function name `yieldOnCost`.

Now, we can use it to calculate YoC of a portfolio:

```
namespace App\Builders\Portfolio;

use App\Models\Holding;
use Illuminate\Database\Eloquent\Builder;

class PortfolioBuilder extends Builder
{
    public function yieldOnCost(): ?float
    {
        $holdings = $this->model->is_aggregate
            ? Holding::whereBelongsTo($this->model->user)->get()
            : $this->model->holdings;

        return $holdings->yieldOnCost();
    }
}
```

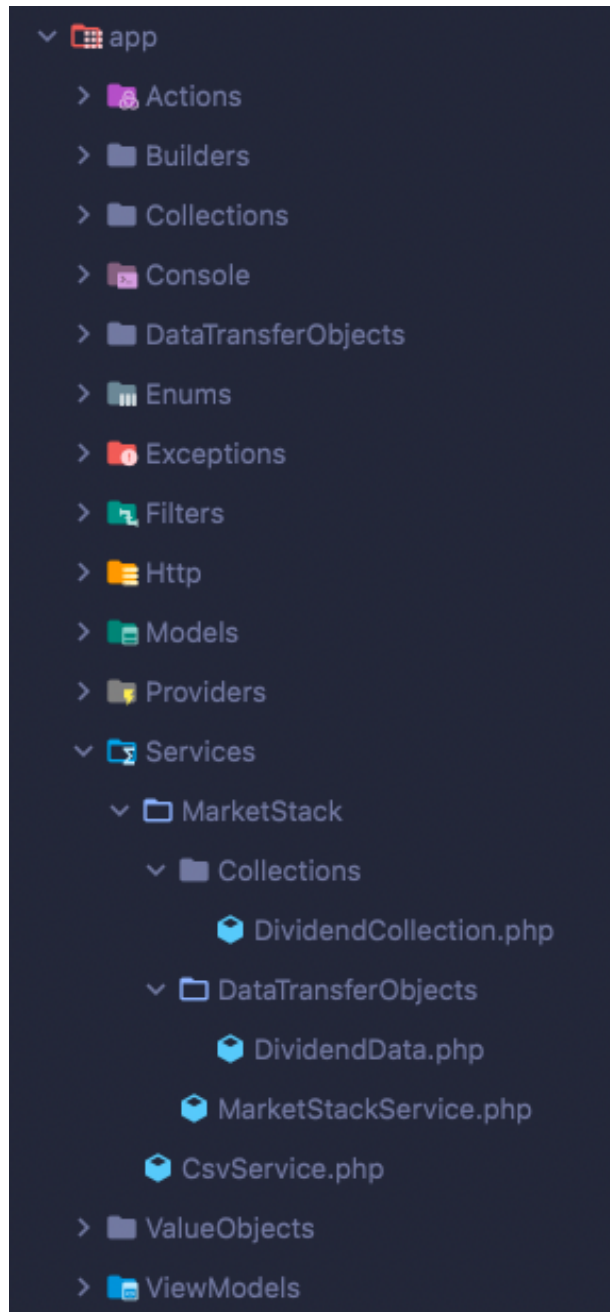
Since a portfolio is just a collection of holdings, we can use the `HoldingCollection` here. The `is_aggregate` means that this portfolio is the "All" for the user, so it contains every other portfolio.

Integrating 3rd Party APIs

When integrating an external service, I like to think about them as "mini-applications" inside my application. So they have a separate folder inside the `App\Services` folder; they have their own classes such as:

- DTOs
- Collections
- Value Objects

This is what I'm talking about:



Each external service has a main class called `xyzService`; in this case, it's `MarketStackService`. This is the class that sends HTTP requests and parses responses. This class usually has several arguments in the constructor:

```
class MarketStackService
{
    public function __construct(
        private readonly string $uri,
        private readonly string $accessKey,
        private readonly int $timeout
    ) {}
}
```

These config values come from the `.env` file, where I always prefix these variables with the service's name:

```
MARKET_STACK_ACCESS_KEY=YOUR_TOKEN_HERE
MARKET_STACK_URI=http://api.marketstack.com/v1/
MARKET_STACK_TIMEOUT=5
```

Laravel includes a `config/services.php` file by default. This can be used to read the external service's config from the env:

```
return [
    'market_stack' => [
        'access_key' => env('MARKET_STACK_ACCESS_KEY'),
        'uri' => env('MARKET_STACK_URI', 'http://api.marketstack.com/v1/'),
        'timeout' => env('MARKET_STACK_TIMEOUT', 5),
    ],
];
```

So in this file, each service has its array key. Notice that the array keys inside `market_stack` are not prefixed.

As the last step, I create a separate service provider for every 3rd party, such as this one:

```
namespace App\Providers;

use App\Services\MarketStack\MarketStackService;
use Illuminate\Support\ServiceProvider;

class MarketStackServiceProvider extends ServiceProvider
{
    public function boot()
    {
        $this->app->singleton(MarketStackService::class, fn () =>
            new MarketStackService(
                config('services.market_stack.uri'),
                config('services.market_stack.access_key'),
                config('services.market_stack.timeout'),
            )
        );
    }
}
```

I construct a new instance of the `MarketStackService` and bind it into the service container in the boot method. Usually, this bind can be a singleton since we only need one instance of `MarketStackService`. Since this class contains scalar values (such as the URI), you must do this step to be able to inject this class like you inject other classes:

```
namespace App\Actions\Holding;

class UpdateMarketValuesAction
{
    public function __construct(
        private readonly MarketStackService $marketStackService
    ) {}
}
```

Laravel can resolve this class only because I created a bind in the service provider. If I miss this step, I'll get an exception because Laravel cannot resolve `string` values in the constructor of the `MarketStackService` class.

After everything is set up correctly, we can write some actual code in the service:

```
class MarketStackService
{
    public function __construct(
        private readonly string $uri,
        private readonly string $accessKey,
        private readonly int $timeout
    ) {}

    public function dividends(string $ticker): DividendCollection
    {
        $response = $this->buildRequest()
            ->get($this->uri . 'dividends', $this->buildQuery($ticker))
            ->throw();

        $items = collect($response->json('data'))
            ->map(fn (array $item) => DividendData::fromArray($item))
            ->toArray();

        return new DividendCollection($items);
    }
}
```

This method will return the dividend payouts for the given ticker symbol. As you can see, I parse the response into a DTO, and I create a new collection from these DTOs.

Working with huge 3rd party responses can be significant pain and a source of many bugs. By using these techniques, you can eliminate many of those bugs.

But don't overdo it:

```
public function price(string $ticker): float
{
    $response = $this->buildRequest()
        ->get($this->uri . 'eod', [
            'limit' => 1,
            ... $this->buildQuery($ticker),
        ])
        ->throw()
        ->json('data');

    return (float) $response[0]['close'];
}
```

In this case, all I need is a simple number. So there's no need to use fancy classes here.

Final Thoughts

Being a small application, it's relatively easy to design and build this portfolio tracker. But as you have seen, there are always several interesting decisions we need to make, and there are always opportunities to improve the code's overall quality.

In this particular case, there were two techniques I can help you with:

- Custom collections.
- Integrating APIs.

I hope you picked some new stuff here that you can use in your day-to-day job. This was our mini case study; I hope you enjoyed it. Feel free to browse the source code and if you have any questions, just reach out to me on [Twitter](#) or e-mail me at martin@martinjoo.dev.